

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	9
2 МЕТОД РЕШЕНИЯ.....	11
3 ОПИСАНИЕ АЛГОРИТМОВ.....	15
3.1 Алгоритм метода set_ready класса cl_base.....	15
3.2 Алгоритм метода get_subobject_on_branch класса cl_base.....	16
3.3 Алгоритм метода get_subobject_on_tree класса cl_base.....	17
3.4 Алгоритм метода print_tree класса cl_base.....	18
3.5 Алгоритм метода print_ready класса cl_base.....	19
3.6 Алгоритм метода build_tree_objects класса cl_application.....	20
3.7 Алгоритм метода exes_app класса cl_application.....	21
3.8 Алгоритм конструктора класса cl_2.....	22
3.9 Алгоритм конструктора класса cl_3.....	23
3.10 Алгоритм конструктора класса cl_4.....	23
3.11 Алгоритм конструктора класса cl_5.....	24
3.12 Алгоритм конструктора класса cl_6.....	24
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	25
5 КОД ПРОГРАММЫ.....	39
5.1 Файл cl_2.cpp.....	39
5.2 Файл cl_2.h.....	39
5.3 Файл cl_3.cpp.....	39
5.4 Файл cl_3.h.....	40
5.5 Файл cl_4.cpp.....	40
5.6 Файл cl_4.h.....	41
5.7 Файл cl_5.cpp.....	41

5.8 Файл cl_5.h.....	41
5.9 Файл cl_6.cpp.....	42
5.10 Файл cl_6.h.....	42
5.11 Файл cl_application.cpp.....	42
5.12 Файл cl_application.h.....	45
5.13 Файл cl_base.cpp.....	45
5.14 Файл cl_base.h.....	50
5.15 Файл main.cpp.....	52
6 ТЕСТИРОВАНИЕ.....	53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	54

1 ПОСТАНОВКА ЗАДАЧИ

Первоначальная сборка системы (дерева иерархии объектов, модели системы) осуществляется исходя из входных данных. Данные вводятся построчно. Первая строка содержит имя корневого объекта (объект приложение). Номер класса корневого объекта 1. Далее, каждая строка входных данных определяет очередной объект, задает его характеристики и расположение на дереве иерархии. Структура данных в строке:

«Наименование головного объекта» «Наименование очередного объекта» «Номер класса принадлежности очередного объекта»

Ввод иерархического дерева завершается, если наименование головного объекта равно «endtree» (в данной строке ввода больше ничего не указывается).

Поиск головного объекта выполняется от последнего созданного объекта. Первоначально последним созданным объектом считается корневой объект. Если для головного объекта обнаруживается дублиаж имени в непосредственно подчиненных объектах, то объект не создается. Если обнаруживается дублиаж имени на дереве иерархии объектов, то объект не создается. Если номер класса объекта задан некорректно, то объект не создается.

Вывод иерархического дерева объектов на консоль.

Внутренняя архитектура (вид иерархического дерева объектов) в большинстве реализованных моделях систем динамически меняется в процессе отработки алгоритма. Вывод текущего дерева объектов является важной задачей, существенно помогая разработчику, особенно на этапе тестирования и отладки программы.

В данной задаче подразумевается, что наименования объектов уникальны. Система содержит объекты пяти классов, не считая корневого. Номера классов: 2,3,4,5,6.

Расширить функциональность базового класса:

- метод поиска объекта на ветке дерева иерархии от текущего по имени (метод возвращает адрес на найденный объект или значение nullptr). Передается один параметр строкового типа, содержит наименование искомого объекта. Если на искомой ветке дерева иерархии наименование объекта не уникально или отсутствует, то возвращает значение nullptr;

- метод поиска объекта на дереве иерархии по имени (метод возвращает адрес на найденный объект или значение nullptr). Передается один параметр строкового типа, содержит наименование искомого объекта. Если на дереве иерархии наименование объекта не уникально или отсутствует, то возвращает значение nullptr;

- метод вывода иерархии объектов (дерева или ветки) от текущего объекта (допускается использовать один целочисленный параметр со значением по-умолчанию);

- метод вывода иерархии объектов (дерева или ветки) и отметок их готовности от текущего объекта (допускается использовать один целочисленный параметр со значением по-умолчанию);

- метод установки готовности объекта, в качестве параметра передается переменная целого типа, содержит номер состояния.

Устаревший метод вывода из задачи KB_1 убрать.

Готовность для каждого объекта устанавливается индивидуально. Готовность задается посредством любого отличного от нуля целого числового значения, которое присваивается свойству состояния объекта. Объект переводится в состояние готовности, если все объекты вверх по иерархии до корневого включены, иначе установка готовности игнорируется. При отключении головного, отключаются все объекты от него по иерархии вниз по ветке. Свойству состояния объекта присваивается значение нуль.

Разработать программу:

1. Построить дерево объектов системы (в методе корневого объекта построения исходного дерева объектов).

2. В методе корневого объекта запуска моделируемой системы реализовать: 2.1. Вывод на консоль иерархического дерева объектов в следующем виде:

```
root
  ob_1
    ob_2
  ob_3
    ob_4
      ob_5
    ob_6
      ob_7
```

где: root - наименование корневого объекта (приложения).

2.2. Переключение готовности объектов согласно входным данным (командам). 2.3. Вывод на консоль иерархического дерева объектов и отметок их готовности в следующем виде:

```
root  is ready
  ob_1  is ready
    ob_2  is ready
  ob_3  is ready
    ob_4 is not ready
      ob_5 is not ready
    ob_6 is ready
      ob_7 is not ready
```

1.1 Описание входных данных

Множество объектов, их характеристики и расположение на дереве иерархии. Последовательность ввода организовано так, что головной объект для очередного вводимого объекта уже присутствует на дереве иерархии объектов.

Первая строка:

«Наименование корневого объекта»

Со второй строки:

```
«Наименование головного объекта» «Наименование очередного объекта» «Номер
класса принадлежности очередного объекта»
. . . . .
endtree
```

Со следующей строки вводятся команды включения или отключения объектов

```
«Наименование объекта» «Номер состояния объекта»
```

Пример ввода:

```
app_root
app_root object_01 3
app_root object_02 2
object_02 object_04 3
object_02 object_05 5
object_01 object_07 2
endtree
app_root 1
object_07 3
object_01 1
object_02 -2
object_04 1
```

1.2 Описание выходных данных

Вывести иерархию объектов в следующем виде:

```
Object tree
«Наименование корневого объекта»
  «Наименование объекта 1»
    «Наименование объекта 2»
      «Наименование объекта 3»
. . . . .
The tree of objects and their readiness
«Наименование корневого объекта» «Отметка готовности»
  «Наименование объекта 1» «Отметка готовности»
    «Наименование объекта 2» «Отметка готовности»
      «Наименование объекта 3» «Отметка готовности»
. . . . .
«Отметка готовности» - равно «is ready» или «is not ready»
```

Отступ каждого уровня иерархии 4 позиции.

Пример вывода:

```
Object tree
app_root
  object_01
    object_07
```

```
    object_02
      object_04
      object_05
The tree of objects and their readiness
app_root is ready
  object_01 is ready
    object_07 is not ready
  object_02 is ready
    object_04 is ready
    object_05 is not ready
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект производный 1 класса `cl_2` предназначен для построения дерева иерархии объектов в количестве согласно вводу;
- объект производный 2 класса `cl_3` предназначен для построения дерева иерархии объектов в количестве согласно вводу;
- объект производный 3 класса `cl_4` предназначен для построения дерева иерархии объектов в количестве согласно вводу;
- объект производный 4 класса `cl_5` предназначен для построения дерева иерархии объектов в количестве согласно вводу;
- объект производный 5 класса `cl_6` предназначен для построения дерева иерархии объектов в количестве согласно вводу.

Класс `cl_base`:

- свойства/поля:
 - поле состояние готовности объекта:
 - наименование — `status`;
 - тип — целочисленный;
 - модификатор доступа — `private`;
- функционал:
 - метод `set_ready` — метод установки состояния готовности для текущего объекта;
 - метод `get_subobject_on_branch` — метод поиска объекта на ветке дерева иерархии от текущего по имени;
 - метод `get_subobject_on_tree` — метод поиска объекта на всём дереве иерархии по имени;
 - метод `print_tree` — метод вывода иерархии объектов (дерева

или ветки) от текущего объекта;

- о метод `print_ready` — метод вывода иерархии объектов и состояний их готовности от текущего объекта.

Класс `cl_application`:

- функционал:
 - о метод `build_tree_objects` — строит исходное дерево иерархии объектов;
 - о метод `exes_app` — запускает систему.

Класс `cl_2`:

- функционал:
 - о метод `cl_2` — конструирует объект второго производного класса путём вызова конструктора базового класса.

Класс `cl_3`:

- функционал:
 - о метод `cl_3` — конструирует объект третьего производного класса путём вызова конструктора базового класса.

Класс `cl_4`:

- функционал:
 - о метод `cl_4` — конструирует объект четвёртого производного класса путём вызова конструктора базового класса.

Класс `cl_5`:

- функционал:
 - о метод `cl_5` — конструирует объект пятого производного класса путём вызова конструктора базового класса.

Класс `cl_6`:

- функционал:
 - о метод `cl_6` — конструирует объект шестого производного

класса путём вызова конструктора базового класса.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base			Базовый класс для всех объектов системы, обеспечивающий возможность построения иерархического дерева объектов, где каждый объект может иметь головной объект и множество подчинённых объектов	
		cl_application	public		2
		cl_2	public		3
		cl_3	public		4
		cl_4	public		5
		cl_5	public		6
		cl_6	public		7
2	cl_application			Класс, производный от базового cl_base, являющийся корневым в дереве иерархии. Обеспечивает построение дерева объектов согласно входным данным и запуск основного алгоритма	
3	cl_2			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии второго класса	

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
4	cl_3			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии третьего класса	
5	cl_4			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии четвёртого класса	
6	cl_5			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии пятого класса	
7	cl_6			Класс, производный от базового cl_base, используется для создания объектов дерева иерархии шестого класса	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `set_ready` класса `cl_base`

Функционал: метод установки состояния готовности для текущего объекта.

Параметры: целочисленный, `state`, новое состояние готовности (0 - выключен, любое ненулевое значение - включён).

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `set_ready` класса `cl_base`

№	Предикат	Действия	№ перехода
1	значение параметра отлично от нуля	инициировать указатель на объект базового класса <code>p</code> значением поля <code>p_head_object</code> текущего объекта	2
		присвоить полю <code>status</code> текущего объекта нулевое значение	6
2	значение указателя <code>p</code> не нулевое		3
			5
3	поле состояния готовности по указателю <code>p</code> равно 0		Ø
			4
4		присвоить указателю <code>p</code> значение результата вызова метода <code>get_head</code> по указателю <code>p</code>	2
5		присвоить полю <code>status</code> текущего объекта значение параметра <code>state</code>	Ø

№	Предикат	Действия	№ перехода
6		инициировать целочисленную переменную i нулевым значением	7
7	значение индекса i меньше размера массива подчинённых объектов	вызов метода set_ready по i-ому указателю из массива адресов подчинённых объектов с аргументом 0	8
			∅
8		инкрементировать i	7

3.2 Алгоритм метода get_subobject_on_branch класса cl_base

Функционал: метод поиска объекта на ветке дерева иерархии от текущего по имени.

Параметры: строка, name, наименование искомого объекта.

Возвращаемое значение: адрес найденного объекта (nullptr при его отсутствии или дуближе).

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода get_subobject_on_branch класса cl_base

№	Предикат	Действия	№ перехода
1		инициировать указатель на объект базового класса p_found нулевым значением (nullptr)	2
2	имя текущего объекта совпадает с искомым	присвоить указателю p_found адрес текущего объекта	3
			3
3		инициировать целочисленную переменную цикла i нулевым значением	4
4	значение индекса i меньше размера массива подчинённых объектов	инициировать указатель на объект базового класса p_result результатом рекурсивного вызова метода get_subobject_on_branch с name в качестве	5

№	Предикат	Действия	№ перехода
		аргумента по i-ому указателю из массива адресов подчинённых объектов	
		вернуть значение указателя p_found	∅
5	в подчинённой ветке найден объект с искомым именем		6
			8
6	искомый объект уже был найден ранее?	вернуть значение нулевого указателя	∅
			7
7		присвоить указателю p_found адрес найденного объекта	8
8		инкрементировать i	4

3.3 Алгоритм метода get_subobject_on_tree класса cl_base

Функционал: метод поиска объекта на всём дереве иерархии по имени.

Параметры: строка, патне, наименование искомого объекта.

Возвращаемое значение: адрес найденного объекта (nullptr при его отсутствии или дуближе).

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода get_subobject_on_tree класса cl_base

№	Предикат	Действия	№ перехода
1		инициировать указатель на объект базового класса root адресом текущего объекта	2
2	у объекта, на который указывает root, есть головной объект	присвоить указателю root адрес его головного объекта	2
		вернуть результат вызова функции	∅

№	Предикат	Действия	№ перехода
		get_subobject_on_branch по указателю root на корневой объект	

3.4 Алгоритм метода print_tree класса cl_base

Функционал: метод вывода иерархии объектов (дерева или ветки) от текущего объекта.

Параметры: целочисленный, depth, текущая глубина отступа (количество уровней вложенности от корня).

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода print_tree класса cl_base

№	Предикат	Действия	№ перехода
1		инициировать целочисленную переменную i нулевым значением	2
2	значение переменной i меньше значения входного параметра depth	вывод на консоль 4 пробелов	3
			4
3		инкрементировать i	2
4		вывод на консоль имени текущего объекта и символ переноса строки	5
5		инициировать целочисленную переменную j нулевым значением	6
6	значение индекса j меньше размера массива подчинённых объектов	вызов метода print_tree рекурсивно по указателю на i-тый элемент массива адресов подчинённых объектов с глубиной отступа depth+1 в качестве аргумента	7

№	Предикат	Действия	№ перехода
			Ø
7		инкрементировать j	6

3.5 Алгоритм метода print_ready класса cl_base

Функционал: метод вывода иерархии объектов и состояний их готовности от текущего объекта.

Параметры: целочисленный, depth, текущая глубина отступа (количество уровней вложенности от корня).

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода print_ready класса cl_base

№	Предикат	Действия	№ перехода
1		инициировать целочисленную переменную i нулевым значением	2
2	значение переменной i меньше значения входного параметра depth	вывод на консоль 4 пробелов	3
			4
3		инкрементировать i	2
4		вывод на консоль имени текущего объекта	5
5	значение поля status текущего объекта не равно нулю	вывод на консоль строки " is ready" и символа переноса строки	6
		вывод на консоль строки " is not ready" и символа переноса строки	6
6		инициировать целочисленную переменную j нулевым значением	7

№	Предикат	Действия	№ перехода
7	значение индекса j меньше размера массива подчинённых объектов	вызов метода print_ready рекурсивно по указателю на i-тый элемент массива адресов подчинённых объектов с глубиной отступа depth+1 в качестве аргумента	8
			∅
8		инкрементировать j	7

3.6 Алгоритм метода build_tree_objects класса cl_application

Функционал: строит исходное дерево иерархии объектов.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода build_tree_objects класса cl_application

№	Предикат	Действия	№ перехода
1		объявить строковые переменные s_head и s_sub	2
2		объявить целочисленную переменную class_number	3
3		считать значение из стандартного потока ввода в переменную s_head	4
4		вызов метода редактирования имени set_name текущего объекта с аргументом s_head	5
5		считать значение из стандартного потока ввода в переменную s_head	6
6	значение переменной s_head равно "endtree"		∅
		считать значение из стандартного потока ввода в переменные s_sub и class_number	7

№	Предикат	Действия	№ перехода
7	на дереве уже есть объект с именем создаваемого		5
			8
8	номер класса меньше 2 или больше 6		5
		инициировать указатель на объект базового класса p_head адресом предполагаемого головного объекта посредством вызова метода get_subobject_on_tree с аргументом s_head	9
9	предполагаемый головной объект не был найден?		5
			10
10	значение переменной class_number равно 2	динамически создать объект класса cl_2 с аргументами p_head и s_sub	5
	значение переменной class_number равно 3	динамически создать объект класса cl_3 с аргументами p_head и s_sub	5
	значение переменной class_number равно 4	динамически создать объект класса cl_4 с аргументами p_head и s_sub	5
	значение переменной class_number равно 5	динамически создать объект класса cl_5 с аргументами p_head и s_sub	5
	значение переменной class_number равно 6	динамически создать объект класса cl_6 с аргументами p_head и s_sub	5
			5

3.7 Алгоритм метода exes_app класса cl_application

Функционал: запускает систему.

Параметры: нет.

Возвращаемое значение: целочисленный - код возврата программы,

определяющий корректность выполнения работы.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *exes_app* класса *cl_application*

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Object tree" и символа переноса строки	2
2		вызов метода <code>print_tree</code> текущего объекта	3
3		объявить строковую переменную <code>object_name</code> для хранения имени объекта	4
4		объявить целочисленную переменную <code>state</code> для хранения значения готовности объекта	5
5	можно ли ввести из стандартного потока ввода значения в переменные <code>object_name</code> и <code>state</code> ?	присвоить указателю на объект базового класса <code>p_object</code> результат вызова метода <code>get_subobject_on_tree</code> текущего объекта с аргументом <code>object_name</code>	6
		вывод на консоль строки "The tree of objects and their readiness" и символа переноса строки	7
6	в дереве есть объект с введённым именем	установить искомому объекту значение готовности равное введённому значению	5
			5
7		вызов метода <code>print_tree</code> текущего объекта	∅

3.8 Алгоритм конструктора класса *cl_2*

Функционал: конструирует объект второго производного класса путём вызова конструктора базового класса.

Параметры: указатель на объект базового класса, `p_head_object`, содержит адрес головного объекта; строка, `s_object_name`, наименование создаваемого объекта.

Алгоритм конструктора представлен в таблице 9.

Таблица 9 – Алгоритм конструктора класса *cl_2*

№	Предикат	Действия	№ перехода
1			Ø

3.9 Алгоритм конструктора класса *cl_3*

Функционал: конструирует объект третьего производного класса путём вызова конструктора базового класса.

Параметры: указатель на объект базового класса, *p_head_object*, содержит адрес головного объекта; строка, *s_object_name*, наименование создаваемого объекта.

Алгоритм конструктора представлен в таблице 10.

Таблица 10 – Алгоритм конструктора класса *cl_3*

№	Предикат	Действия	№ перехода
1			Ø

3.10 Алгоритм конструктора класса *cl_4*

Функционал: конструирует объект четвёртого производного класса путём вызова конструктора базового класса.

Параметры: указатель на объект базового класса, *p_head_object*, содержит адрес головного объекта; строка, *s_object_name*, наименование создаваемого объекта.

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса cl_4

№	Предикат	Действия	№ перехода
1			Ø

3.11 Алгоритм конструктора класса cl_5

Функционал: конструирует объект пятого производного класса путём вызова конструктора базового класса.

Параметры: указатель на объект базового класса, p_head_object, содержит адрес головного объекта; строка, s_object_name, наименование создаваемого объекта.

Алгоритм конструктора представлен в таблице 12.

Таблица 12 – Алгоритм конструктора класса cl_5

№	Предикат	Действия	№ перехода
1			Ø

3.12 Алгоритм конструктора класса cl_6

Функционал: конструирует объект шестого производного класса путём вызова конструктора базового класса.

Параметры: указатель на объект базового класса, p_head_object, содержит адрес головного объекта; строка, s_object_name, наименование создаваемого объекта.

Алгоритм конструктора представлен в таблице 13.

Таблица 13 – Алгоритм конструктора класса cl_6

№	Предикат	Действия	№ перехода
1			Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-14.

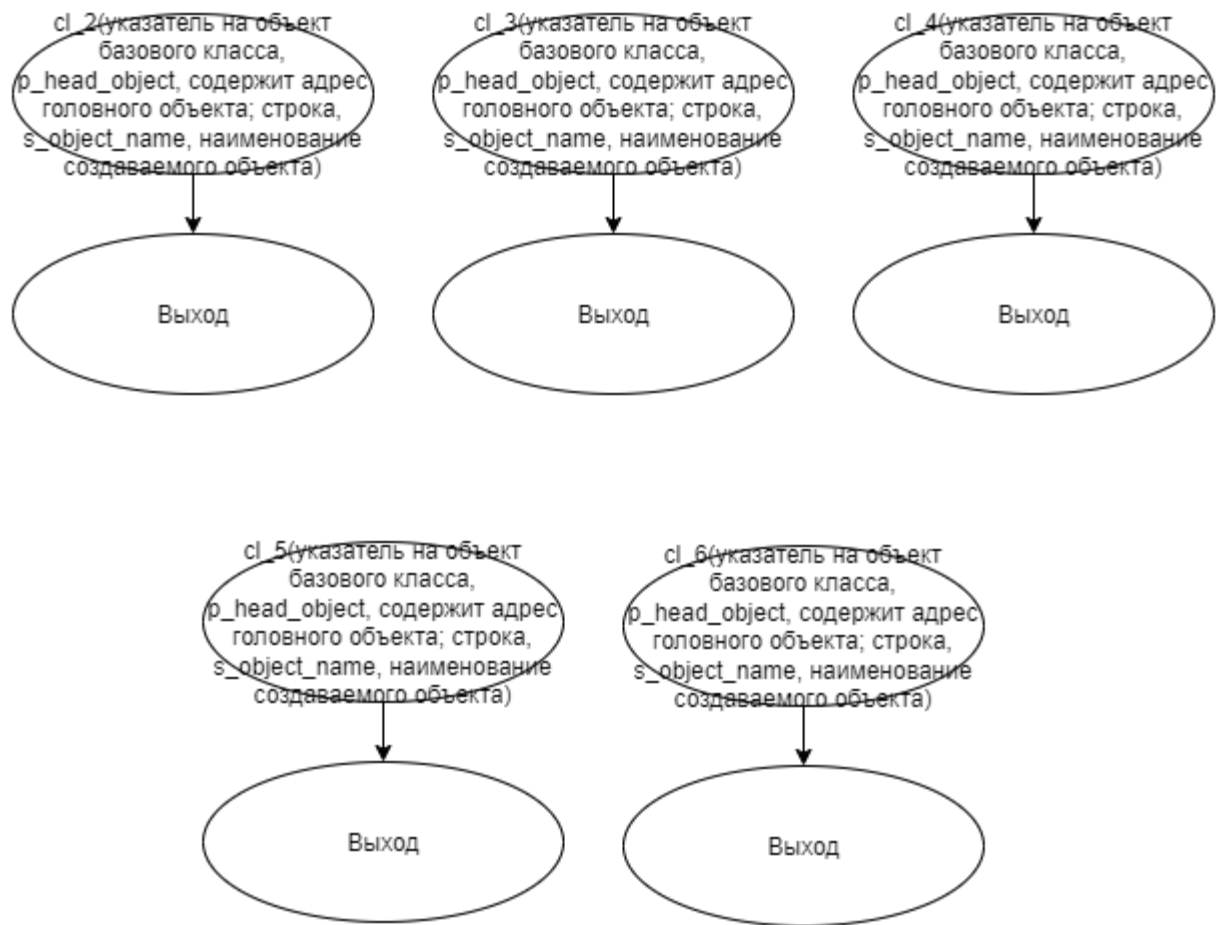


Рисунок 1 – Блок-схема алгоритма

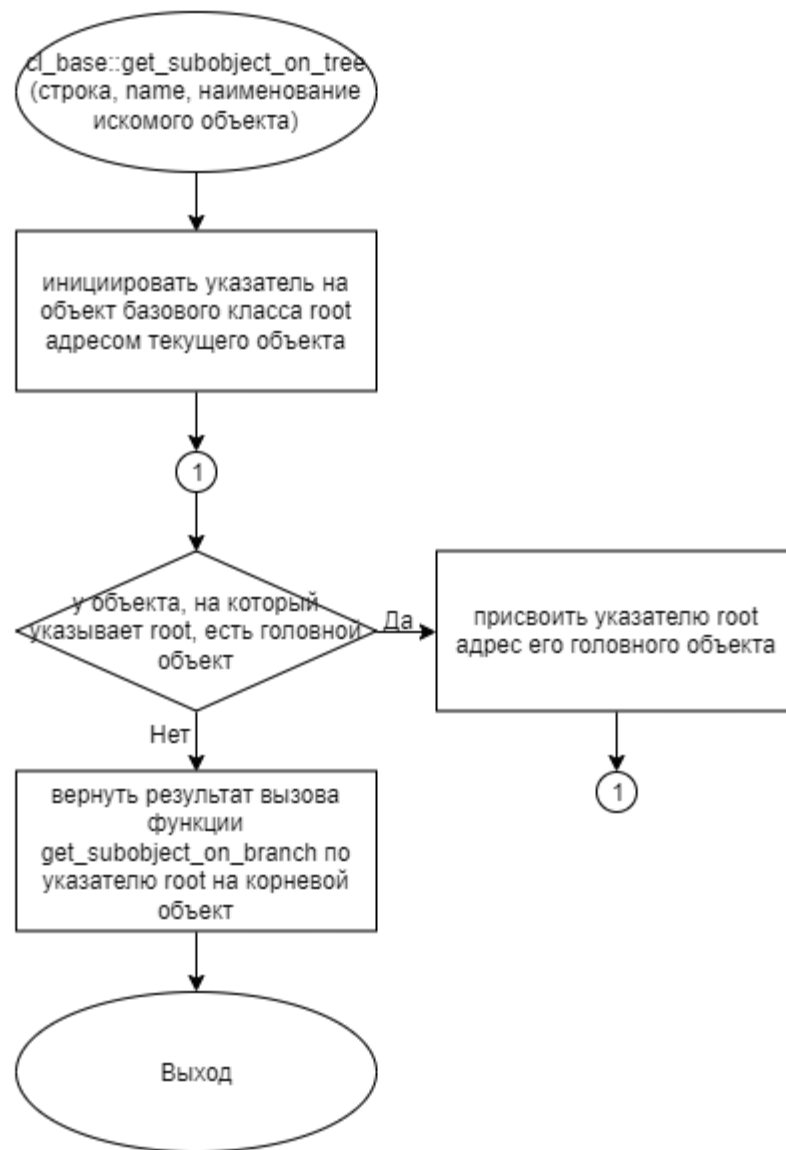


Рисунок 2 – Блок-схема алгоритма

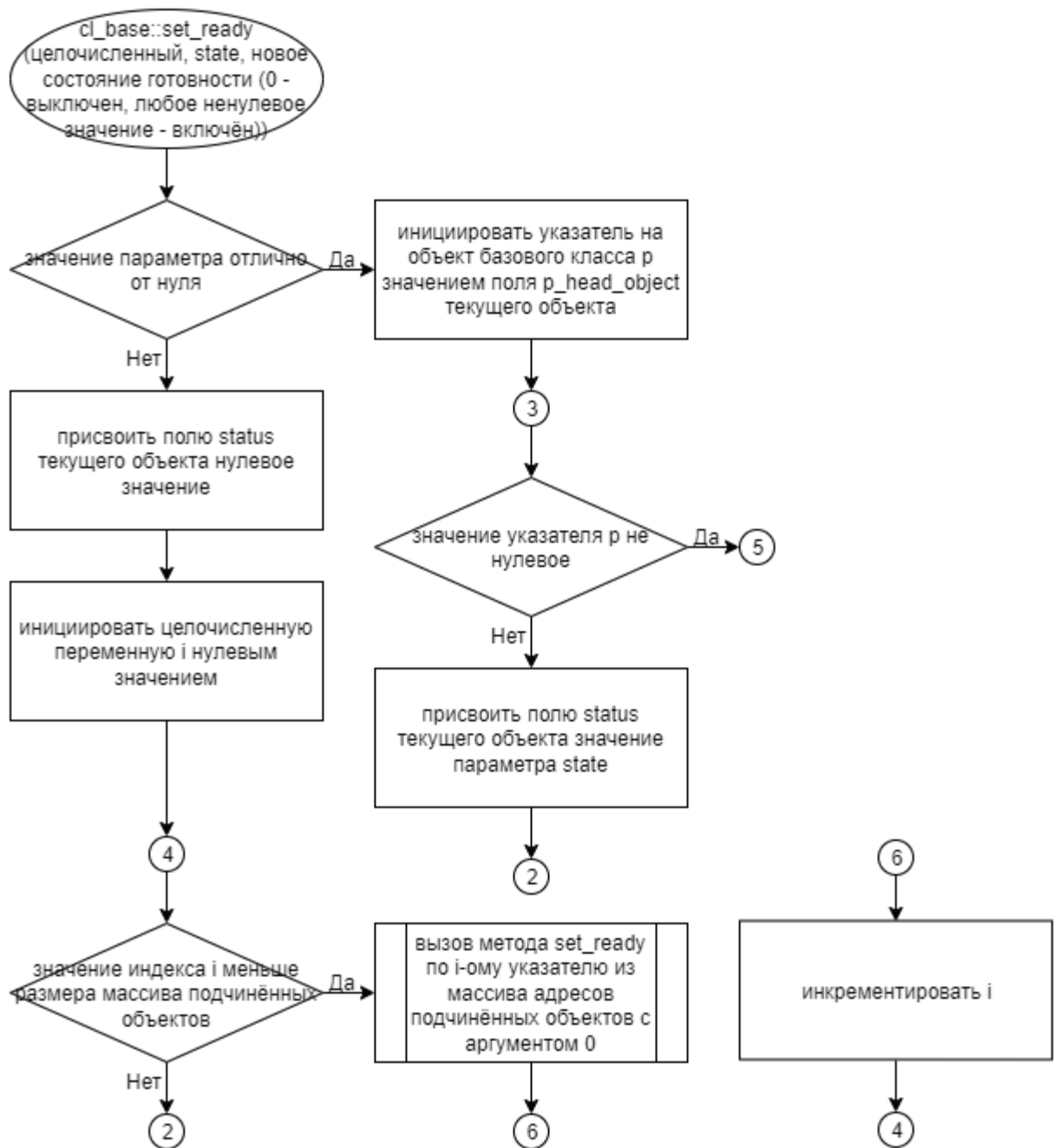


Рисунок 3 – Блок-схема алгоритма

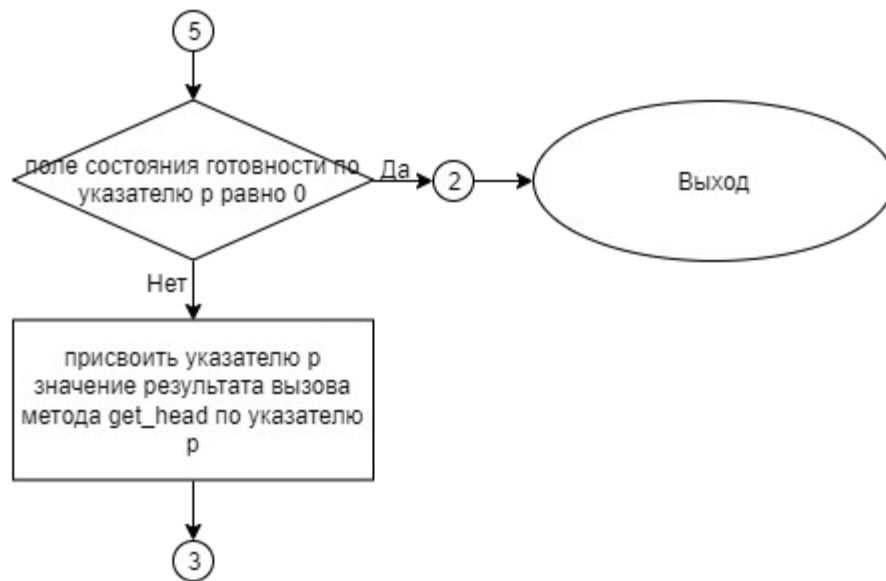


Рисунок 4 – Блок-схема алгоритма

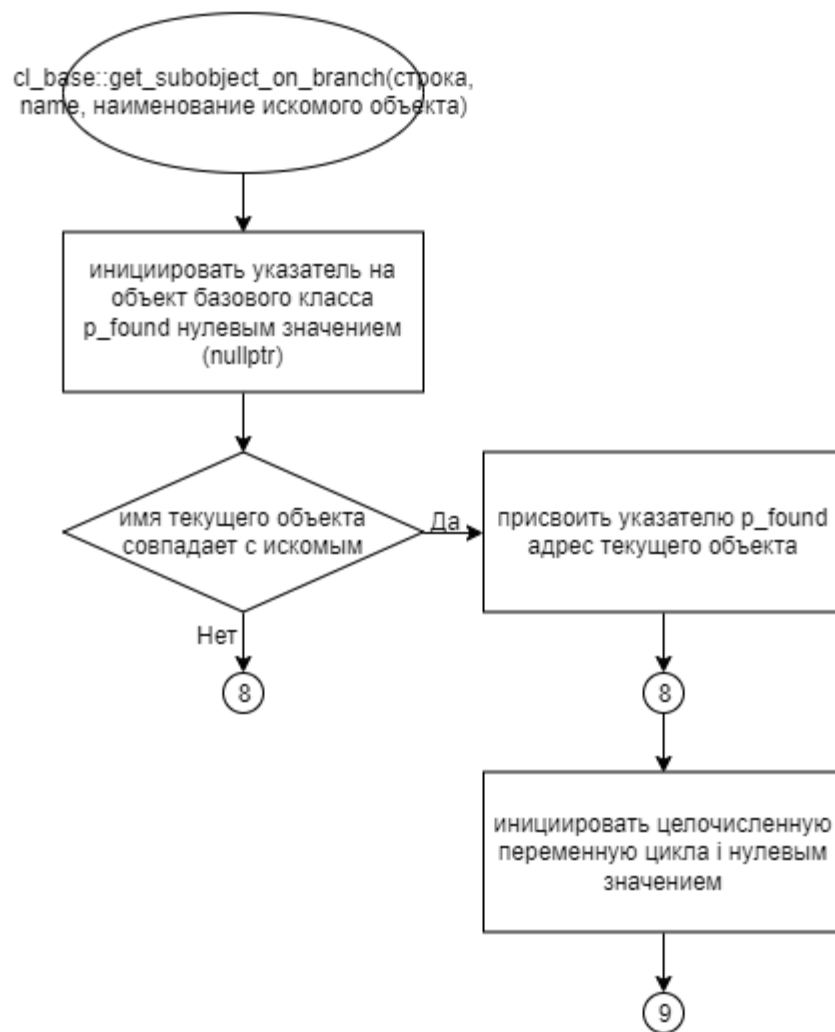


Рисунок 5 – Блок-схема алгоритма

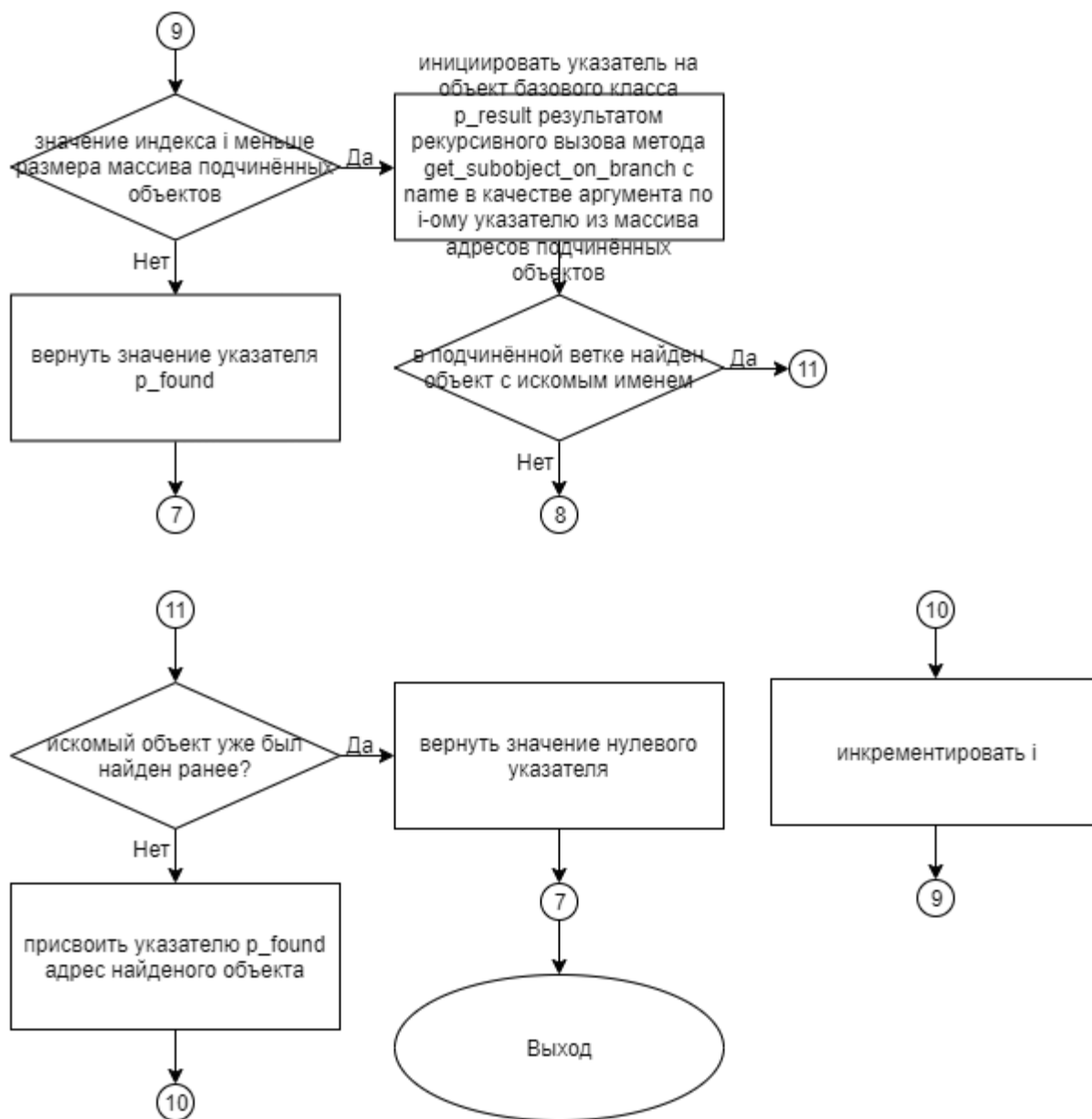


Рисунок 6 – Блок-схема алгоритма

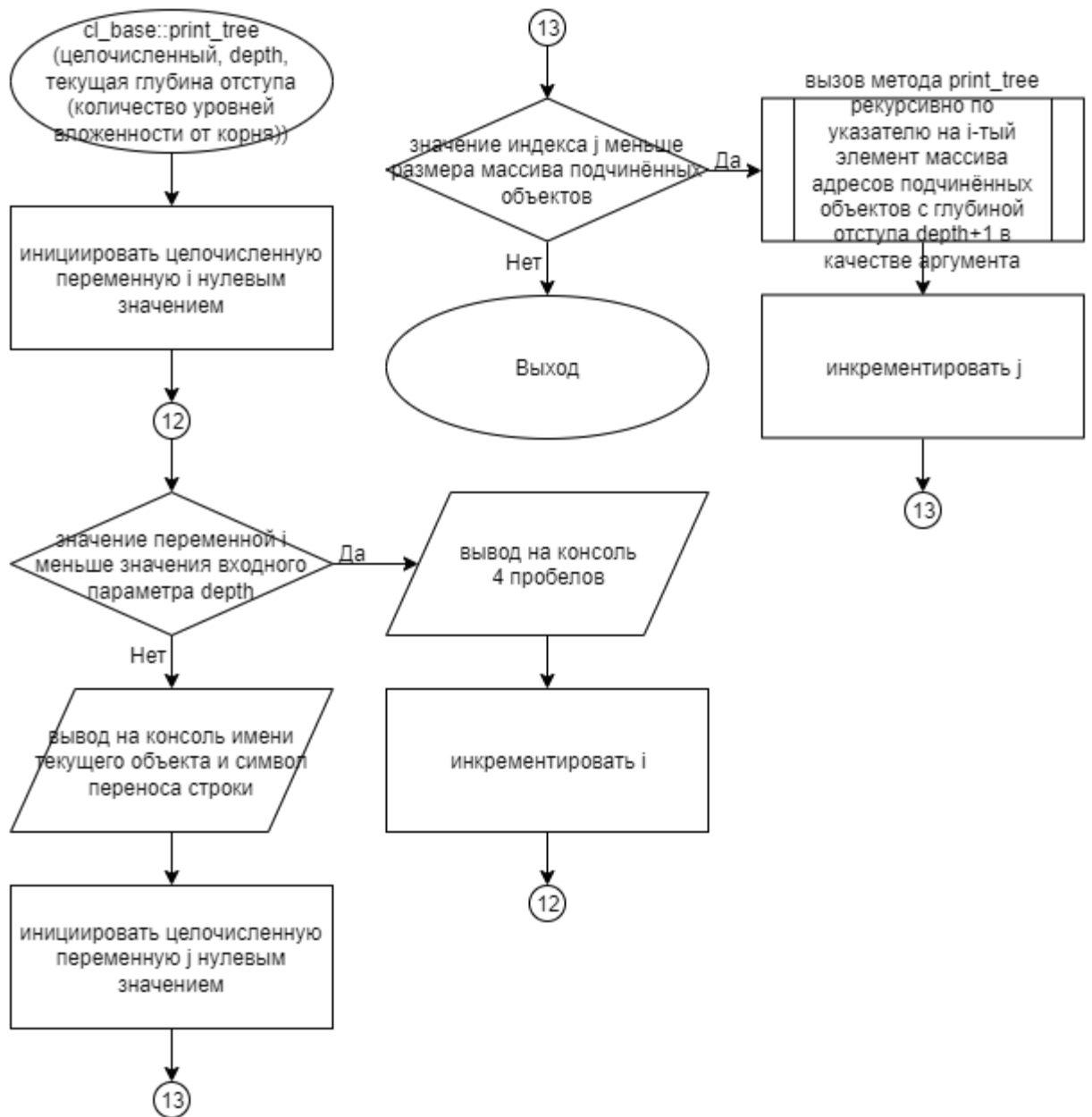


Рисунок 7 – Блок-схема алгоритма

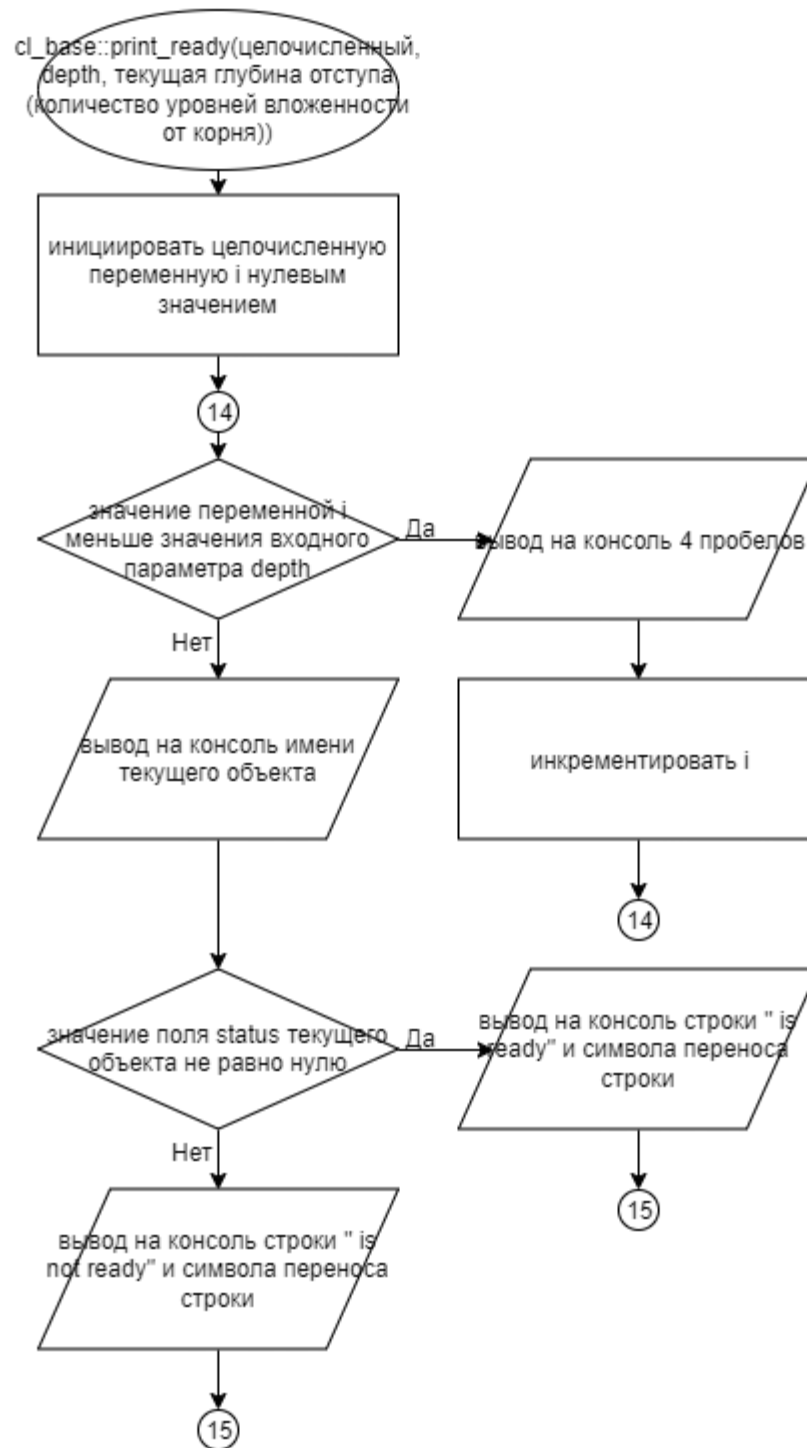


Рисунок 8 – Блок-схема алгоритма

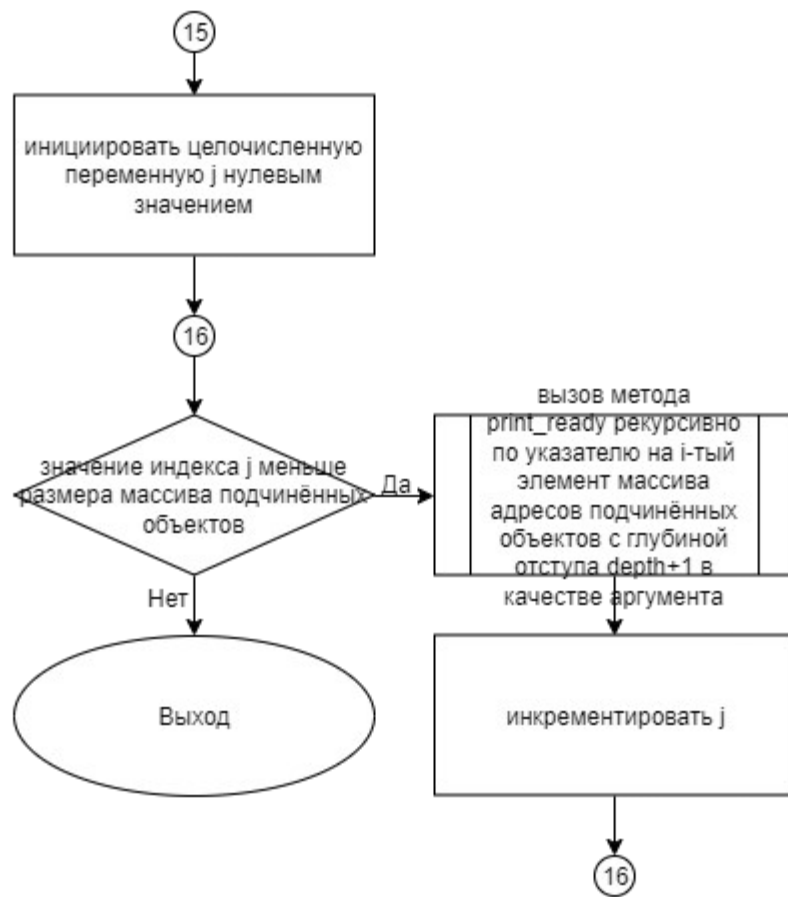


Рисунок 9 – Блок-схема алгоритма

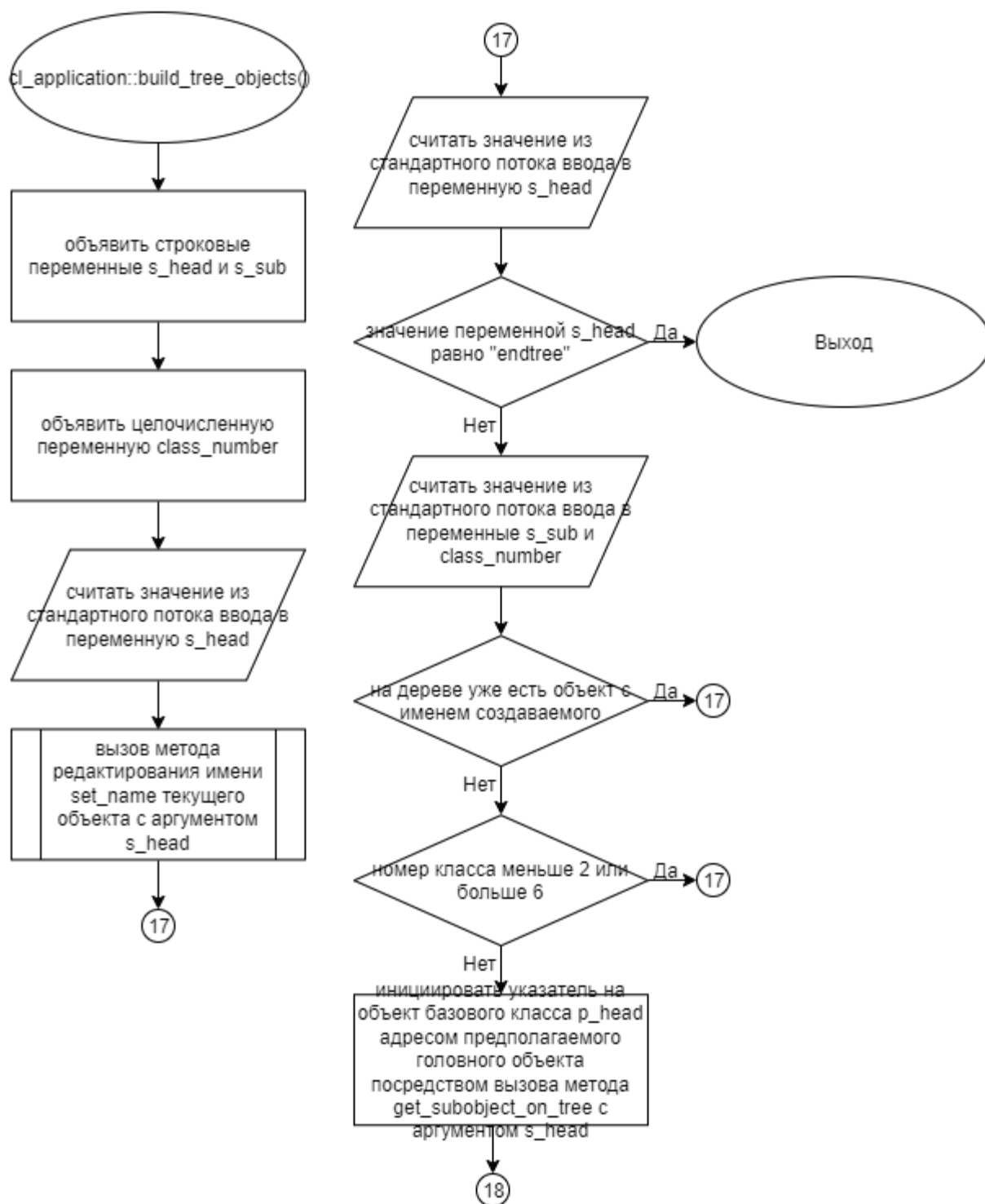


Рисунок 10 – Блок-схема алгоритма

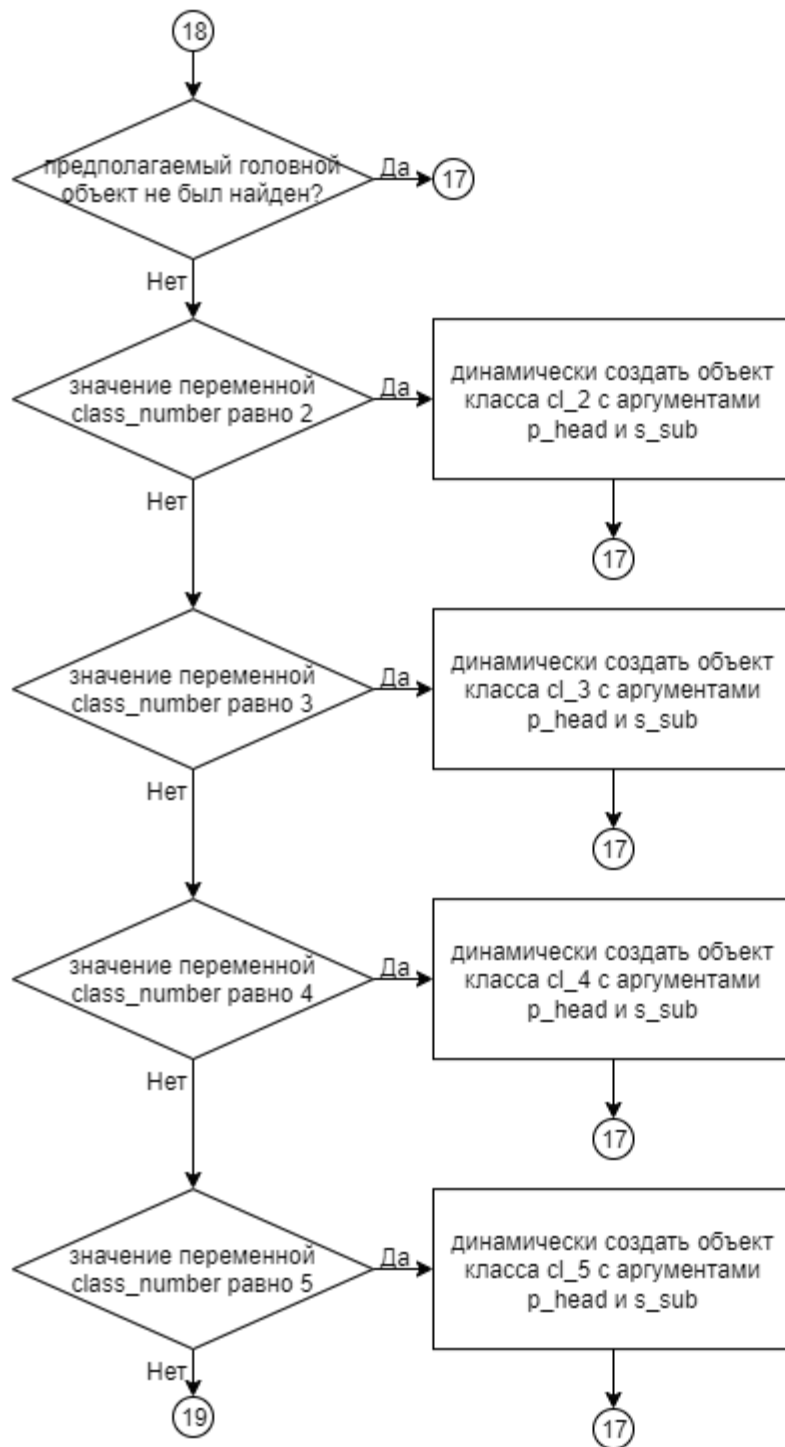


Рисунок 11 – Блок-схема алгоритма

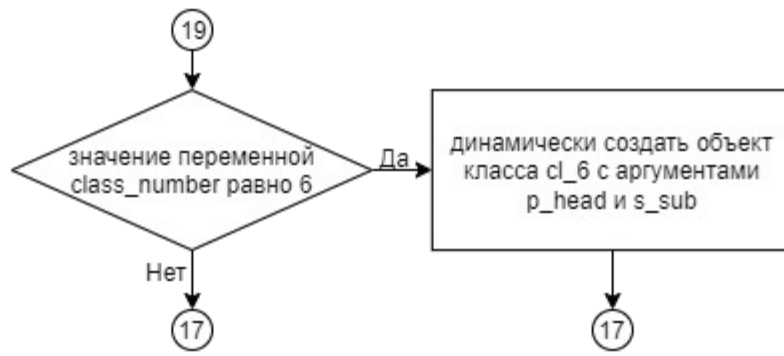


Рисунок 12 – Блок-схема алгоритма



Рисунок 13 – Блок-схема алгоритма

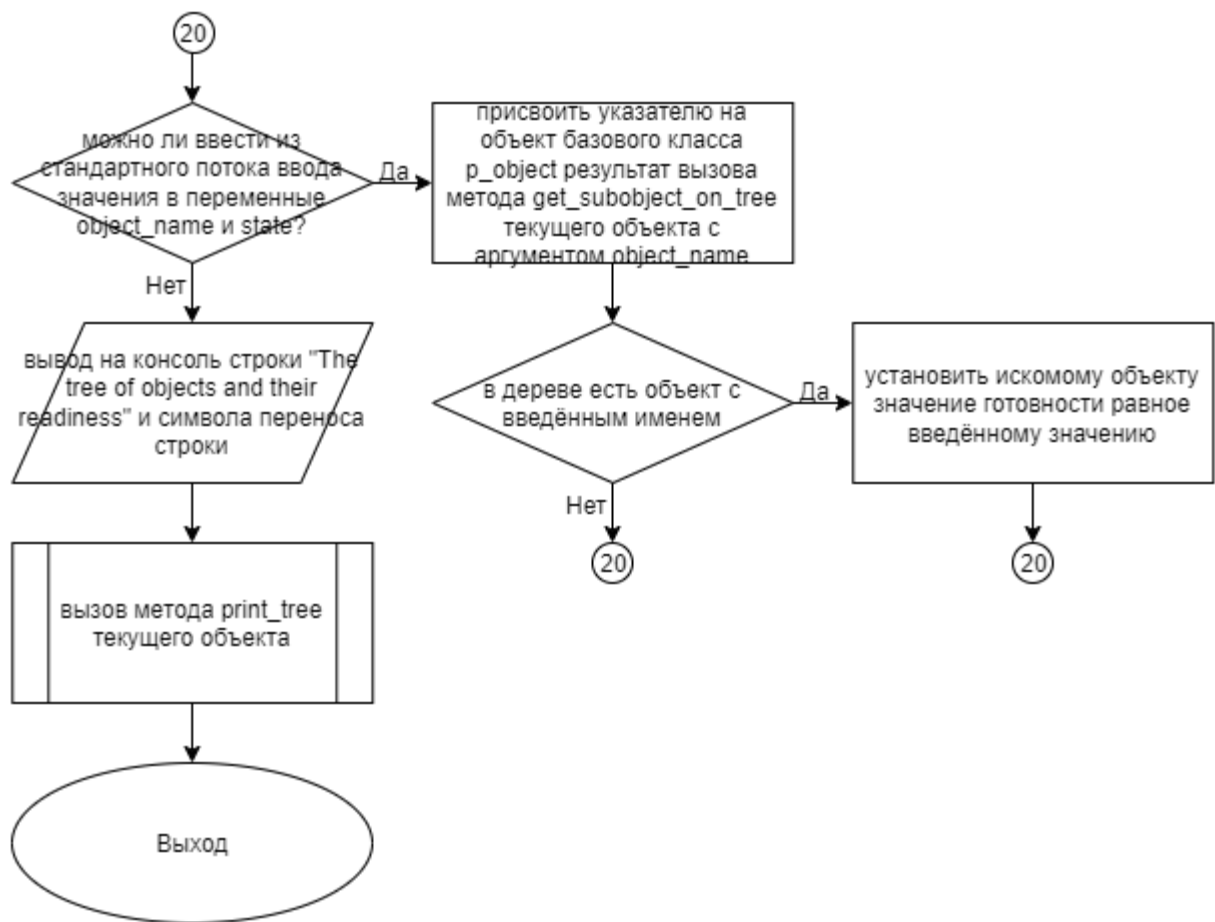


Рисунок 14 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_2.cpp

Листинг 1 – cl_2.cpp

```
#include "cl_2.h"

cl_2 :: cl_2 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект
производного класса с указателем на головной объект и наименованием путём
вызова конструктора базового класса
```

5.2 Файл cl_2.h

Листинг 2 – cl_2.h

```
#ifndef __CL_2__H
#define __CL_2__H

#include "cl_base.h"

class cl_2 : public cl_base
{
public:
    cl_2 ( cl_base * p_head_object, string s_object_name ); //
параметризированный конструктор
};

#endif
```

5.3 Файл cl_3.cpp

Листинг 3 – cl_3.cpp

```
#include "cl_3.h"
```

```
cl_3 :: cl_3 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.4 Файл cl_3.h

Листинг 4 – cl_3.h

```
#ifndef __CL_3__H  
#define __CL_3__H  
  
#include "cl_base.h"  
  
class cl_3 : public cl_base  
{  
public:  
    cl_3 ( cl_base * p_head_object, string s_object_name ); //  
параметризованный конструктор  
};  
  
#endif
```

5.5 Файл cl_4.cpp

Листинг 5 – cl_4.cpp

```
#include "cl_4.h"  
  
cl_4 :: cl_4 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.6 Файл cl_4.h

Листинг 6 – cl_4.h

```
#ifndef __CL_4__H
#define __CL_4__H

#include "cl_base.h"

class cl_4 : public cl_base
{
public:
    cl_4 (    cl_base *    p_head_object,    string    s_object_name    );    //
    параметризованный конструктор
};

#endif
```

5.7 Файл cl_5.cpp

Листинг 7 – cl_5.cpp

```
#include "cl_5.h"

cl_5 :: cl_5 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }    // конструирует объект
производного класса с указателем на головной объект и наименованием путём
вызова конструктора базового класса
```

5.8 Файл cl_5.h

Листинг 8 – cl_5.h

```
#ifndef __CL_5__H
#define __CL_5__H

#include "cl_base.h"

class cl_5 : public cl_base
{
public:
    cl_5 (    cl_base *    p_head_object,    string    s_object_name    );    //
    параметризованный конструктор
```

```
};  
  
#endif
```

5.9 Файл cl_6.cpp

Листинг 9 – cl_6.cpp

```
#include "cl_6.h"  
  
cl_6 :: cl_6 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.10 Файл cl_6.h

Листинг 10 – cl_6.h

```
#ifndef __CL_6__H  
#define __CL_6__H  
  
#include "cl_base.h"  
  
class cl_6 : public cl_base  
{  
public:  
    cl_6 ( cl_base * p_head_object, string s_object_name ); //  
параметризированный конструктор  
};  
  
#endif
```

5.11 Файл cl_application.cpp

Листинг 11 – cl_application.cpp

```
#include "cl_application.h"  
#include "cl_2.h"
```

```

#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
#include <iostream>
#include <string>
#include <vector>
using namespace std;

cl_application :: cl_application ( cl_base * p_head_object, string
s_object_name )
: cl_base ( p_head_object, s_object_name ) { } // создаёт корневой
объект приложения, предварительно вызывая конструктор базового класса

void cl_application :: build_tree_objects ( ) // строит исходное
дерево иерархии объектов
{
    string s_head, s_sub; // имена головного и
    подчинённого объектов
    int class_number; // объявл. целочисленной
    переменной class_number

    cin >> s_head; // считать значение из
    стандартного потока ввода в переменную s_head
    this -> set_name ( s_head ); // устанавливаем имя
    текущего объекта значением s_head вызовом метода set_name

    while ( true ) // начало бесконечного цикла
    {
        cin >> s_head; // считать значение из
        стандартного потока ввода в переменную s_head

        if ( s_head == "endtree" ) // значение переменной
        s_head равно "endtree"
        {
            break; // прервать выполнение цикла
            - завершение построения дерева
        }

        cin >> s_sub >> class_number; // считать значение из
        стандартного потока ввода в переменные s_sub и class_number

        if ( get_subobject_on_tree ( s_sub ) != nullptr )
        { // на дереве уже есть объект с
        именем создаваемого
            continue; // обнаружен дуближ имени на
            дереве - объект не создаётся, переход к следующей итерации
        }

        if ( class_number < 2 || class_number > 6 ) // номер класса введён
        некорректно ( допустимые значения от 2 до 6 )
        {
            continue; // номер задан некорректно
        }

        // инициировать указатель p_head

```

```

адресом предполагаемого головного объекта
    cl_base * p_head = get_subobject_on_tree ( s_head );
    if ( p_head == nullptr )                // предполагаемый головной
объект не был найден?
    {
        continue;                          // объект не создаётся
    }

    if ( class_number == 2 )
    {
        new cl_2 ( p_head, s_sub );          // динамически создать
объект класса cl_2 с введёнными значениями в качестве аргументов
    }
    else if ( class_number == 3 )
    {
        new cl_3 ( p_head, s_sub );          // динамически создать
объект класса cl_3 с введёнными значениями в качестве аргументов
    }
    else if ( class_number == 4 )
    {
        new cl_4 ( p_head, s_sub );          // динамически создать
объект класса cl_4 с введёнными значениями в качестве аргументов
    }
    else if ( class_number == 5 )
    {
        new cl_5 ( p_head, s_sub );          // динамически создать
объект класса cl_5 с введёнными значениями в качестве аргументов
    }
    else if ( class_number == 6 )
    {
        new cl_6 ( p_head, s_sub );          // динамически создать
объект класса cl_6 с введёнными значениями в качестве аргументов
    }
}

int cl_application :: exec_app ( )           // запускает
систему
{
    cout << "Object tree" << endl;          // вывод заголовка
    this -> print_tree ( );                 // отображение
иерархии

    string object_name;                     // объявл. строковой
переменной object_name для хранения имени объекта
    int state;                             // объявл. целочисленной
переменной state для хранения значения готовности объекта

    while ( cin >> object_name >> state )   // можно ли ввести
object_name и state?
    {
        cl_base * p_object = this -> get_subobject_on_tree ( object_name );

        if ( p_object != nullptr )         // в дереве есть
объект с введённым именем

```

```

        {
            p_object -> set_ready ( state );          // установить    искомому
объекту значение готовности равное введённому значению
        }
    }

    cout << "The tree of objects and their readiness" << endl;
    this -> print_ready ( );                          //            отображение
итоговой иерархии

    return 0;
}

```

5.12 Файл cl_application.h

Листинг 12 – cl_application.h

```

#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"

class cl_application : public cl_base
{
public:
    cl_application ( cl_base * p_head_object, std :: string s_object_name =
"Application" ); // параметризованный конструктор
    void          build_tree_objects          (          );
// строит исходное дерево иерархии объектов: чтение входных данных, создание
объектов с проверкой уникальности имён и правил добавления
    int          exes_app          (          );
// запускает систему: чтение команд редактирования имён объектов по уровням
и индексам, изменение имён и вывод обновлённого дерева
};

#endif

```

5.13 Файл cl_base.cpp

Листинг 13 – cl_base.cpp

```

#include "cl_base.h"
#include <iostream>
using namespace std;

cl_base :: cl_base ( cl_base * p_head_object, string s_object_name ) //

```

```

параметризированный конструктор
{
    this -> s_object_name = s_object_name;
    // хранение наименования объекта
    this -> p_head_object = p_head_object;
    // хранение адреса на головной объект

    if ( p_head_object ) {
        // есть ли головной объект?
        p_head_object -> subordinate_objects.push_back ( this );
        // добавляем адрес текущего объекта в список адресов подчинённых объектов
        // головного объекта
    }
}

cl_base :: ~cl_base ( )
    // деструктор
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
    {
        // значение индекса i меньше размера вектора подчинённых объектов?
        delete subordinate_objects [ i ];
        // удалить динамически созданный объект, адрес которого хранится в i-
        // том элементе массива подчинённых объектов текущего объекта
    }
}

bool cl_base :: set_name ( string new_name )
    // редактирует имя объекта с проверкой отсутствия дуближа среди
    // подчинённых объектов у головного
{
    if ( this -> p_head_object != nullptr )
        // есть ли головной объект?
    {
        for ( int i = 0; i < p_head_object -> get_subordinate_count ( ); i++ )
        {
            // значение индекса i меньше количества подчинённых объектов головного объекта
            if ( p_head_object -> subordinate_objects [ i ] -> get_name ( ) ==
            new_name )
            {
                // проверяем уникальность имени
                return false;
                // есть другой подчинённый с таким же именем
            }
        }
    }

    this -> s_object_name = new_name;
    // установка нового имени
    return true;
    // имя успешно изменено
}

string cl_base :: get_name ( )
    // возвращает наименование объекта

```

```

{
    return this -> s_object_name;
    // текущее наименование объекта
}

cl_base * cl_base :: get_head()
    // возвращает адрес головного объекта текущего объекта
{
    return this -> p_head_object;
    // адрес головного объекта текущего объекта
}

int cl_base :: get_subordinate_count ( )
    // возвращает количество подчинённых объектов текущего объекта
{
    return (int) this -> subordinate_objects.size ( );
    // текущее количество подчинённых объектов
}

cl_base * cl_base :: get_subobject_by_name ( string name )
    // возвращает адрес подчинённого объекта по его имени (если не найден -
    // возвращает nullptr)
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )          //
        значение индекса i меньше количества элементов в списке подчинённых объектов
        текущего объекта
        {
            if ( subordinate_objects [ i ] -> s_object_name == name )          //
            {
                имя текущего подчинённого объекта совпадает со значением входного параметра
                name
                return subordinate_objects [ i ];
                // вернуть значение i-того указателя из массива подчинённых объектов
            }
        }

    return nullptr;
    // объект не найден
}

cl_base * cl_base :: get_subobject_by_index ( int index )
    // возвращает адрес подчинённого объекта по его индексу (от 1 до
    // количества подчинённых)
{
    if ( index < 1 || index > this -> get_subordinate_count ( ) )          //
        проверка на корректность индекса
        {
            return nullptr;
            // объект не найден
        }

    return subordinate_objects [ index - 1 ];
    // такой подчинённый существует
}

cl_base * cl_base :: get_subobject_on_branch ( string name )          //

```

```

метод поиска объекта на ветке дерева иерархии от текущего по имени
{
    cl_base * p_found = nullptr;
    // иниц. указатель p_found нулевым значением
    if ( this -> s_object_name == name )
        // имя текущего объекта совпадает с искомым
    {
        p_found = this;
        // текущий объект является искомым
    }

    for ( int i = 0; i < get_subordinate_count ( ); i++ )
        // обход всех подчинённых объектов текущего объекта
    {
        // вызов метода get_subobject_on_branch рекурсивно для подчинённого
        cl_base * p_result = subordinate_objects [ i ] ->
        get_subobject_on_branch ( name );

        if ( p_result != nullptr )
            // в подчинённой ветке найден объект с искомым именем
        {
            if ( p_found != nullptr )
                // искомый объект уже был найден ранее?
            {
                return nullptr;
                // обнаружен дуближ имени на ветке
            }

            p_found = p_result;
            // присвоить указателю p_found адрес найденного объекта
        }
    }

    return p_found;
    // вернуть адрес найденного объекта (или nullptr, если объект не
    найден)
}

cl_base * cl_base :: get_subobject_on_tree ( string name )
// метод поиска объекта на всём дереве иерархии по имени
{
    cl_base * root = this;
    // иниц. указатель root значением адреса текущего объекта

    while ( root -> get_head() != nullptr )
        // у объекта, на который указывает root, есть головной объект
    {
        root = root -> get_head ( );
        // присвоить указателю root адрес его головного объекта
    }

    return root -> get_subobject_on_branch ( name );
    // вернуть результат вызова функции get_url_on_branch по указателю root
    на корневой объект root
}

```

```

void cl_base :: print_tree ( int depth )
    // метод вывода иерархии объектов (дерева или ветки) от текущего
    объекта
{
    for ( int i = 0; i < depth; i++ )
        // вывод отступа: по 4 пробела на каждый уровень вложенности
        {
            cout << "    ";
            // вывод на консоль 4 пробелов
        }

    cout << this -> get_name ( ) << endl;
    // вывод имени текущего объекта и переход на новую строку
    for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )
    {
        subordinate_objects [ j ] -> print_tree ( depth + 1 );
        // вызов метода print_tree рекурсивно по указателю на i-тый элемент массива
        // подчинённых объектов с глубиной отступа +1 в качестве аргумента
    }
}

void cl_base :: print_ready ( int depth )
    // метод вывода иерархии объектов и состояний их готовности от текущего
    объекта
{
    for ( int i = 0; i < depth; i++ )
        // значение переменной i меньше значения входного параметра depth
        {
            cout << "    ";
            // вывод на консоль 4 пробелов
        }

    cout << this -> get_name ( );
    // вывод на консоль имени текущего объекта

    if ( this -> status != 0 )
        // значение поля status текущего объекта не равно нулю
        {
            cout << " is ready" << endl;
            // объект готов
        }
    else
    {
        cout << " is not ready" << endl;
        // объект не готов
    }

    for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )
        // значение индекса j меньше размера массива подчинённых объектов
        {
            subordinate_objects [ j ] -> print_ready ( depth + 1 );
            // вызов метода print_ready рекурсивно по указателю на i-тый элемент
            // массива подчинённых объектов с глубиной отступа +1 в качестве аргумента
        }
}

```

```

}

void cl_base :: set_ready ( int state )
{
    if ( state != 0 )
        // включение объекта
        {
            проверка готовности для всех вышестоящих объектов до корня
            cl_base * p = this -> p_head_object;
            // начать с головного объекта, текущий объект не проверяется

            while ( p != nullptr )
            // пока не достигнут корень
            {
                if ( p -> status == 0 )
                    // обнаружен вышестоящий объект в выключенном состоянии
                    {
                        return;
                    }
                // установка готовности для текущего объекта невозможна
            }

            p = p -> get_head ( );
            // переход к головному объекту (подъём на уровне выше)
        }

        this -> status = state;
        // установка состояния готовности текущего объекта
    }
    else
        // выключение объекта (передано нулевое значение)
        {
            this -> status = 0;
            // сброс состояния готовности текущего объекта

            for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
                // цикл по всем подчинённым объектам
                {
                    subordinate_objects [ i ] -> set_ready ( 0 );
                    // рекурсивный вызов метода set_ready для отключения подчинённых объектов
                }
        }
}

```

5.14 Файл cl_base.h

Листинг 14 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H

```

```

#include <string>
#include <vector>
using namespace std;

class cl_base
{
private:
    string s_object_name;
        // хранение наименования объекта
    cl_base * p_head_object;
        // хранение адреса на головной объект
    vector < cl_base * > subordinate_objects;
        // динамический массив адресов подчинённых объектов
    int status;
        // состояние готовности объекта

public:
    cl_base ( cl_base* p_head_object, string s_object_name = "Base_object" );
        // параметризованный конструктор
    ~cl_base ( );
        // деструктор для удаления текущего объекта и всех его подчинённых
        // объектов

    bool set_name ( string new_name );
        // редактирует имя объекта с проверкой отсутствия дуближа среди
        // подчинённых объектов у головного
    string get_name ( );
        // возвращает наименование объекта
    cl_base * get_head ( );
        // возвращает адрес головного объекта текущего объекта
    int get_subordinate_count ( );
        // возвращает количество подчинённых объектов текущего объекта
    cl_base * get_subobject_by_name ( string name );
        // возвращает адрес подчинённого объекта по его имени (если не
        // найден - возвращает nullptr)
    cl_base * get_subobject_by_index ( int index );
        // возвращает адрес подчинённого объекта по его индексу (от 1 до
        // количества подчинённых)

    cl_base * get_subobject_on_branch ( string name );
        // метод поиска объекта на ветке дерева иерархии от текущего по
        // имени
    cl_base * get_subobject_on_tree ( string name );
        // метод поиска объекта на всём дереве иерархии по имени
    void print_tree ( int depth = 0 );
        // метод вывода иерархии объектов (дерева или ветки) от текущего
        // объекта
    void print_ready ( int depth = 0 );
        // метод вывода иерархии объектов и состояний их готовности от
        // текущего объекта
    void set_ready ( int state );
        // метод установки состояния готовности для текущего объекта
};

#endif

```

5.15 Файл main.cpp

Листинг 15 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>

#include "cl_application.h"
using namespace std;

int main()
{
    cl_application ob_cl_application ( nullptr ); // создание корневого
    объекта
    ob_cl_application.build_tree_objects ( );      // конструирование системы,
    построение дерева объектов
    return ob_cl_application.exec_app ( );        // запуск системы
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 14.

Таблица 14 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
app_root app_root object_01 3 app_root object_02 2 object_02 object_04 3 object_02 object_05 5 object_01 object_07 2 endtree app_root 1 object_07 3 object_01 1 object_02 -2 object_04 1	Object tree app_root object_01 object_07 object_02 object_04 object_05 The tree of objects and their readiness app_root is ready object_01 is ready object_07 is not ready object_02 is ready object_04 is ready object_05 is not ready	Object tree app_root object_01 object_07 object_02 object_04 object_05 The tree of objects and their readiness app_root is ready object_01 is ready object_07 is not ready object_02 is ready object_04 is ready object_05 is not ready
root root child1 2 root child2 3 root child1 4 endtree	Object tree root child1 child2 The tree of objects and their readiness root is ready child1 is not ready child2 is not ready	Object tree root child1 child2 The tree of objects and their readiness root is ready child1 is not ready child2 is not ready

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).